

최단 경로 알고리즘 정리

1. 다익스트라 알고리즘 (Dijkstra's Algorithm)

1-1. 개요

- **목적:** 하나의 시작 정점 v 에서 그래프 내 다른 모든 정점까지의 최단 경로 길이를 찾는 알고리즘
- **특징:** 각 단계마다 최단 경로가 확정된 정점 집합 S 에, 가장 짧은 거리를 가진 정점을 추가하며 진행

1-2. 기본 개념

- 시작 정점 v 에서 자기 자신까지의 거리는 0으로 초기화
- 나머지 정점까지의 거리는 무한대(∞)로 초기화
- 각 단계에서 집합 S 에 속하지 않은 정점 중, 최단 거리 배열에서 가장 작은 값을 가진 정점 u 를 선택해 S 에 추가
- 선택된 u 를 통해 인접한 정점 w 까지의 거리 값을 갱신

1-3. 동작 원리

- 정점 u 를 통해 정점 w 로 가는 거리

$$\text{distance}[w] = \min(\text{distance}[w], \text{distance}[u] + \text{weight}(u, w))$$

- 이 과정을 반복하여 모든 정점까지의 최단 경로를 확정

1-4. 예시

단계	집합 S	distance 배열 (정점별 최단 거리)	선택된 정점
0	{0}	0, 7, ∞ , ∞ , 3, 10, ∞	시작 정점 0
1	{0, 4}	0, 5, ∞ , 14, 3, 6, 8	정점 4 선택
2	{0, 4, 1}	0, 5, 9, 10, 3, 6, 8	정점 1 선택
3	{0, 4, 1, 6}	0, 5, 9, 8, 3, 6, 8	정점 6 선택
...	...	...	...

- 예) 시작점 0에서 4를 거쳐 1로 가는 경로가 5로 갱신됨 ($0 \rightarrow 4(3) + 4 \rightarrow 1(2)$)

1-5. 주요 함수 설명

```
min(distance[u], distance[w], weight(u, w))
```

- $distance[u]$: 시작점에서 새로 집합에 추가된 정점 u 까지의 최단 거리
- $distance[w]$: 시작점에서 아직 집합에 없는 정점 w 까지의 현재까지 알려진 최단 거리
- $weight(u, w)$: u 에서 w 로 가는 간선의 가중치

1-6. 시간 복잡도

- 단순 구현: $O(V^2)$ (정점 수 V)
- 우선순위 큐 사용 시: $O(E \log V)$ (간선 수 E)

2. 플로이드 알고리즘 (Floyd's Algorithm)

2-1. 개요

- **목적**: 모든 정점 쌍 간 최단 경로를 구하는 알고리즘
- 삼중 반복문을 통해 모든 경로에 대해 점진적으로 최단 경로를 갱신

2-2. 초기 설정

- $A[i][j] = 0$ if $i = j$
- $A[i][j] = \text{weight}(i, j)$ if 간선 존재
- $A[i][j] = \infty$ if 간선 없음

2-3. 동작 원리

- 각 정점 k 를 경유지로 가정하고,
- 다음 점화식을 통해 최단 경로 갱신

$$A[i][j] = \min(A[i][j], A[i][k] + A[k][j])$$

2-4. 알고리즘 구조 (의사코드)

```
for k in 1 to n:
  for i in 1 to n:
    for j in 1 to n:
      A[i][j] = min(A[i][j], A[i][k] + A[k][j])
```

2-5. 특징 및 시간 복잡도

- 시간 복잡도: $O(n^3)$
- 모든 정점 쌍 간의 최단 경로를 한 번에 계산
- 다익스트라와 달리 음수 간선도 처리 가능 (단, 음수 사이클은 안됨)

3. 요약

알고리즘	목적	시간 복잡도	특징
다익스트라	단일 출발지 최단 경로 계산	$O(V^2)$ / $O(E \log V)$	탐욕법 기반, 가중치가 음수가 아닌 그래프에서 사용
플로이드	모든 정점 쌍 간 최단 경로 계산	$O(V^3)$	모든 경로를 한 번에 계산, 음수 간선 처리 가능